AIM

Adaptive Isolating Model Compression Algorithm

Canonical Specification — Version 17

Reference implementation: aim3_c | March 2026

This document is the canonical specification for AIM version 17, derived directly from the aim3_c C implementation (aim3_encode.c, aim3_decode.c, recursive.c, ans.c, flags.c, bit_utils.c). It supersedes the Python-reference v16 specification. Where this document diverges from the source code, the source code is authoritative.

1. Overview

AIM is a lossless data compression algorithm. At each layer it selects one bit plane, separates the input into a flag bitset and an aligned stream, compresses the flag bitset with the best available format, and encodes the aligned stream with the best available entropy backend. This can be applied recursively up to eight layers. In auto mode both a single-pass and a recursive container are produced; the smaller one is emitted.

Property	Value
Magic bytes	0x41 0x49 0x4D 0x33 ("AIM3")
Container header	64 bytes fixed (both versions)
Container versions	0x09 = single-pass | 0x0A = recursive
Max recursive layers	8
Flag formats (race)	4: gap+gz (0), bitset+gz (1), Elias-Fano (2), Gamma-RLE (3)
Backends (race)	5: gzip (0), ANS-0 (1), ANS-1 (2), ANS-2d (4), ANS-stride (5)
ANS symbol alphabet	[0, 127] — remapped before encode; unmapped after decode
Gzip input	Raw aligned bytes, pre-remap (gzip backend only)
Min n for recursive	8 (recursive skipped when n < 8)
Integrity check	SHA-256 of original data at header offset 32
Legacy versions decoded	0x02, 0x04 through 0x09

2. Container Header (64 bytes)

Both container versions use the same 64-byte fixed header. The interpretation of fields at offsets 5, 7, 16, and 24 differs between versions.

Offset	Size	Field	Description
0	4 B	magic	0x41 0x49 0x4D 0x33
4	1 B	version	0x09 = single-pass, 0x0A = recursive
5	1 B	field_5	v0x09: target_bit (0-7, bit plane stripped). v0x0A: n_layers (1-8, layers used).
6	1 B	flag_mode	Flag format ID of outermost layer (0-3).
7	1 B	backend	v0x09: winning backend ID (0, 1, 2, 4, or 5). v0x0A: always 5.
8	8 B	orig_size	uint64 big-endian — original uncompressed byte count.
16	8 B	flag_len	uint64 big-endian — byte length of flag block. v0x0A: 0 (no outer flag block).
24	8 B	aln_len	uint64 big-endian — byte length of aligned payload (v0x09) or recursive payload (v0x0A).
32	32 B	sha256	SHA-256 digest of the original uncompressed data.

> *Known issue in aim3_encode.c: the current v0x0A encoder writes the recursive payload length into bytes 20-23 (low 4 bytes of the flag_len field) and places sha256 at offset 24 instead of 32, corrupting aln_len and the digest location. The decoder expects flag_len=0, aln_len=rec_payload_len, sha256 at offset 32. Correct implementations must match the decoder's expectation as documented above.*

Payload at offset 64: flag_block (flag_len bytes) immediately followed by aligned or recursive payload (aln_len bytes).

3. Single-Pass Mode (v0x09)

3.1 Encode

- **1. SHA-256:** Compute digest of input (streaming via sha256_file when src_path given).
- **2. Bit selection:** Try all 8 bit positions. For each, cost = encode_flags_best(flag_bs) + gzip(aligned). Select lowest cost; lowest index on tie. Streaming popcount when src_path given.
- **3. bit_clear:** Strip target_bit from every byte producing aligned[] and flag_bs[].
- **4. Flag race:** Encode flag_bs with all applicable formats; keep smallest (Section 6).
- **5. Remap:** remap_to_128(aligned, target_bit) -> syms[] in [0,127]. Used by ANS backends only.
- **6. Backend race:** GZIP encodes raw aligned[]; ANS backends encode syms[]. Keep smallest (Section 7).
- **7. Pack:** Emit 64-byte header + flag_block + best_payload.

3.2 Decode

- 1. Parse 64-byte header. flag_block at offset 64; aln_block at 64 + flag_len.
- 2. decode_flags(flag_block, orig_size, flag_mode) -> flag_bs[].
- 3. If GZIP: gzip_decompress(aln_block) -> aligned[]. Otherwise: ans_decode(aln_block) -> syms[], then unmap_from_128(syms, target_bit) -> aligned[].
- 4. reconstruct(aligned, orig_size, flag_bs, target_bit) -> result.
- 5. Verify SHA-256 of result against header offset 32.

4. Recursive Mode (v0x0A)

The recursive encoder applies bit-plane extraction repeatedly. At each layer it competes ANS-stride encoding of the residual against recursing one layer deeper, taking the smaller result. The decision is local and greedy at every layer.

4.1 Halt Codes

Code	Name	Trigger	Child Payload
0	HALT_RECURSE	Default. Recursion continues.	Next layer record.
2	HALT_ZERO	all_zero (all syms == 0) OR all_same (all syms identical). Both cases use code 2 in the current encoder.	Empty (child_len=0). Decoder fills aligned[] with 0x00.
3	HALT_ONE	Defined and decoded (fill=0x7F) but never written by the current encoder — encoder uses HALT_ZERO for all_same.	Empty (child_len=0). Decoder fills aligned[] with 0x7F.
4	HALT_ANS	len(ans_stride_payload) + LAYER_HDR (19 B) < len(recursive_child).	ANS-stride payload (Section 8).

4.2 encode_layer

```
encode_layer(data, n, depth, parent_aligned_to_free):
  1. sweep(data, n) -> best_bit              # sparsest; lowest index on tie
  2. bit_clear(data, n, best_bit) -> aligned[], flag_bs[]
     free parent_aligned_to_free             # keeps peak memory at O(n)
  3. encode_flags_best(flag_bs, n) -> flag_block, flag_mode
  4. remap_to_128(aligned, best_bit) -> syms[]
  5. all_zero = all(syms==0);  all_same = all(syms==syms[0])
  6. if not trivial: ans_stride_encode(syms) -> ans_buf  (else ans_ok=-1)
     free syms
  7. if depth+1 < 8 and n >= 8 and not trivial:
         encode_layer(aligned, n, depth+1, &aligned) -> child_buf
     free aligned (no-op if child already freed it at step 2)
  8. Halt decision:
       all_zero or all_same             -> HALT_ZERO,   child=None
       child_buf < ans_buf + 19 B       -> HALT_RECURSE, child=child_buf
```

```
      else                                -> HALT_ANS,    child=ans_buf
  9. Emit layer record (see Section 5)
```

4.3 decode_layer

```
decode_layer(payload, orig_n):
  parse 19-byte header -> bit, flag_mode, fl, halt, cl
  flag_raw = payload[19 : 19+fl]
  child    = payload[19+fl : 19+fl+cl]
  flag_bs  = decode_flags(flag_raw, orig_n, flag_mode)
  if halt == HALT_ZERO:     syms = bytes(orig_n)          # fill 0x00
  if halt == HALT_ONE:      syms = bytes([0x7F] * orig_n) # fill 0x7F
  if halt == HALT_ANS:      syms = ans_stride_decode(child, orig_n)
  if halt == HALT_RECURSE:  syms = decode_layer(child, orig_n)
  aligned = unmap_from_128(syms, bit)
  return reconstruct(aligned, orig_n, flag_bs, bit)
```

5. Recursive Layer Record (LAYER_HDR = 19 bytes)

Offset	Size	Field	Description
0	1 B	bit_stripped	Bit position cleared at this layer (0-7).
1	1 B	flag_mode	Flag format used (0-3).
2	8 B	flag_len	uint64 big-endian — byte length of flag_block.
10	1 B	halt_code	0=HALT_RECURSE, 2=HALT_ZERO, 3=HALT_ONE, 4=HALT_ANS.
11	8 B	child_len	uint64 big-endian — byte length of child payload (0 for HALT_ZERO/ONE).
19	flag_len B	flag_block	Compressed flag bitset.
19+flag_len	child_len B	child	HALT_RECURSE: next layer. HALT_ANS: ANS-stride payload. HALT_ZERO/ONE: absent.

6. Flag Compression Race

The flag bitset is uint8_t[(n+7)/8]. Bit i (0-based, little-endian within each byte) is set iff input position i had the target bit set. encode_flags_best tries all applicable formats and keeps the smallest.

ID	Name	Density gate	Description
0	FLAG_MODE_GAP_GZ	density < 0.40 (BITSET_THRESHOLD)	Gap-encoded position list, gzip-compressed. Two-pass: first finds max_gap to choose encoding width. uint8 gaps (max_gap<=255), uint16 BE (<=65534), or uint16 BE with 0xFFFF sentinels for larger. 4-byte header uint32 BE with flag bits: bit31=two-byte mode, bit30=sentinel mode.

ID	Name	Density gate	Description
1	FLAG_MODE_BITSET	Always	Raw flag_bs gzip-compressed. Universal fallback.
2	FLAG_MODE_EF	Always	Elias-Fano coding of sorted flag positions.
3	FLAG_MODE_RLE	Always	Elias-gamma run-length coding of the bitset.

7. Aligned-Data Backends

GZIP encodes raw aligned bytes (pre-remap). All four ANS backends encode syms[] — remapped [0,127] symbols. On decode, ANS output passes through unmap_from_128 before flag reconstruction. Backend ID 3 is reserved/unused.

ID	Name	Input	Notes
0	BACKEND_GZIP	Raw aligned	zlib/gzip level 9.
1	BACKEND_ANS0	syms [0,127]	rANS order-0. One global frequency table.
2	BACKEND_ANS1	syms [0,127]	rANS order-1. Per-predecessor context. Context = syms[i-1]. Requires >= MIN_CTX_O1 (64) observations.
3	(reserved)	—	Not used.
4	BACKEND_ANS2D	syms [0,127]	rANS order-2 differential. Two-symbol predecessor context. Requires >= MIN_CTX_O2 (128) observations.
5	BACKEND_ANS_STRIDE	syms [0,127]	rANS order-1, stride-k predecessor. k auto-selected. Context for i < k uses 0. See Section 8.

8. ANS-Stride (BACKEND_ANS_STRIDE = 5)

8.1 Stride Selection

```
STRIDE_CANDIDATES = [1, 2, 3, 4, 6, 8, 12, 16]
STRIDE_SAMPLE     = 1,000,000  (probe first 1M remapped symbols)

H(k) = conditional entropy H(X_i | X_{i-k})  over probe window
     = -sum_{c,s}  P(ctx=c) * P(s|c) * log2(P(s|c))

k_opt = argmin_k  H(k)

# 5% relative gain threshold:
if H(k=1) == 0  or  H(k_opt) >= H(k=1) * 0.95:  k_opt = 1
```

Threshold is 5% relative gain (not the absolute 0.05 bits/sym from v16). Context for i < k uses 0 (not a sentinel 256 as in v16 — symbols are in [0,127]).

8.2 Wire Format

Field	Size	Description
k	1 B	Selected stride (1, 2, 3, 4, 6, 8, 12, or 16).
global_table	variable	Packed freq table. Format: uint16 BE n_nonzero, then n_nonzero x (uint8 sym, uint16 BE freq).
ctx_mask	16 B	128-bit bitmask. Bit c (byte c>>3, bit c&7) set iff per-ctx table for c exists (ctx_n[c] >= 64).
per_ctx_tables	variable	One packed freq table per set bit in ctx_mask, ascending c=0..127.
first_sym	1 B	syms[0] stored verbatim (not ANS-encoded).
state	4 B BE	Final ANS state after encoding symbols n-1 downto 1.
renorm_bytes	N B	Renorm bytes emitted during encode, LSB-first. Decoder reverses before consuming.

8.3 Encode / Decode Loops

```
# Encode (reverse order, i = n-1 downto 1):
x = ANS_L
for i = n-1 downto 1:
    ctx = syms[i-k]  if i >= k  else 0
    t   = ctx_tables[ctx]  if valid  else global_table
    x   = encode_sym(x, syms[i], t, &byte_buf)
emit: k || global_table || ctx_mask || per_ctx_tables || syms[0] || x(BE4) || byte_buf

# Decode (forward order):
rev = reverse(renorm_bytes);  out[0] = first_sym;  x = state
for i = 1 to n-1:
    ctx   = out[i-k]  if i >= k  else 0
    t     = ctx_tables[ctx]  if valid  else global_table
    out[i] = decode_sym(x, t, &rev)
```

9. rANS Codec Details

9.1 Constants

```
ANS_M_BITS=14   ANS_M=16384   ANS_L=16384   ANS_B=256   MIN_CTX_O1=64   MIN_CTX_O2=128
```

9.2 Frequency Table Normalisation (scale_freqs)

```
for s in 0..127:
    freq[s] = max(1, round(raw[s] * ANS_M / n_obs))  if raw[s] > 0  else 0
delta = ANS_M - sum(freq)
# Sort observed symbols by raw count descending.
# Round-robin increment/decrement until delta == 0.
cum[0]=0;  cum[s+1] = cum[s] + freq[s]
slots[cum[s]..cum[s+1]-1] = s  for each observed s
```

9.3 Single-Symbol Encode / Decode

```
# Encode symbol s into state x (emitting renorm bytes LSB-first):
limit = (ANS_L / ANS_M) * ANS_B * freq[s]
while x >= limit:  emit (x & 0xFF);  x >>= 8
```

```
x = (x / freq[s]) * ANS_M + cum[s] + (x % freq[s])

# Decode symbol from state x (consuming reversed byte stream):
slot = x % ANS_M
s    = slots[slot]
x    = freq[s] * (x >> ANS_M_BITS) + slot - cum[s]
while x < ANS_L and bytes remain:  x = (x << 8) | next_reversed_byte
```

10. Core Operations

10.1 bit_clear(data, n, bit, aligned_out, flag_bs)

```
mask = 1 << bit
for i in 0..n-1:
    if data[i] & mask:  flag_bs[i>>3] |= 1 << (i & 7)
    aligned_out[i] = data[i] & ~mask
```

10.2 reconstruct(aligned, n, flag_bs, bit, result)

```
memcpy(result, aligned, n)
mask = 1 << bit
for i in 0..n-1:
    if (flag_bs[i>>3] >> (i & 7)) & 1:  result[i] |= mask
```

10.3 remap_to_128(aligned, n, bit, out)

Packs [0,255] values (bit already cleared) into [0,127]:

```
lo = (1 << bit) - 1;  hi_xor = 0x7F ^ lo
out[i] = (aligned[i] & lo) | ((aligned[i] >> 1) & hi_xor)
```

10.4 unmap_from_128(syms, n, bit, out) — inverse

```
lo = (1 << bit) - 1;  hi_mask = 0x7F ^ lo
out[i] = (syms[i] & lo) | ((syms[i] & hi_mask) << 1)
```

10.5 sweep(data, n) — sparsest bit

```
counts[8] = {0}
for each byte v:  for b in 0..7:  if v & (1<<b): counts[b]++
return lowest b with minimum counts[b]  (ties: lowest index wins)
```

Used by the recursive encoder. The single-pass encoder uses cost-based selection instead (Section 3.1, step 2).

11. Integrity Verification

SHA-256 at header offset 32 covers the original uncompressed data. Decoders must verify by default; the verify parameter to aim3_decode() disables it. For streaming encode, SHA-256 is computed in a separate sequential pass over the file before the main encode loop.

```
sha256_buf(decoded_data, decoded_len, actual)
if memcmp(actual, container + 32, 32) != 0:  return IntegrityError
```

12. Edge Cases

Input	Behaviour
n = 0	Valid container. Single-pass, empty gzip payload.
n < 8	Recursive encoder skipped. Single-pass used directly.
All-zero bytes	Recursive: HALT_ZERO at depth 0 (all_zero=true), child_len=0. Single-pass: normal gzip of aligned stream.
All identical bytes	Recursive: HALT_ZERO at depth 0 (all_same=true), child_len=0. HALT_ONE (code 3) is never written.
Already compressed/random	ANS-stride gain < 5%; k_opt=1. Gzip typically wins backend race.

13. Constants Quick Reference

Constant	Value	Role
AIM3_MAGIC	"AIM3" / 0x41494D33	Container magic bytes 0-3
AIM3_VERSION	0x09	Single-pass version byte
AIM3_VERSION_RECURSIVE	0x0A	Recursive version byte
AIM3_HEADER_SIZE	64	Fixed header length (bytes)
AIM3_GZ_LEVEL	9	zlib compression level
BITSET_THRESHOLD	0.40	Flag density above which gap+gz is skipped
ANS_M_BITS / ANS_M	14 / 16384	rANS normalisation interval
ANS_L	16384	rANS lower renorm bound
ANS_B	256	rANS output byte alphabet
MIN_CTX_O1	64	Min obs for ANS-1 per-context table
MIN_CTX_O2	128	Min obs for ANS-2d per-context table
MAX_RECURSIVE_LAYERS	8	Maximum recursive depth
LAYER_HDR	19	Recursive layer fixed header (bytes)
STRIDE_CANDIDATES	{1,2,3,4,6,8,12,16}	ANS-stride k search set
STRIDE_SAMPLE	1,000,000	Max symbols probed for stride selection